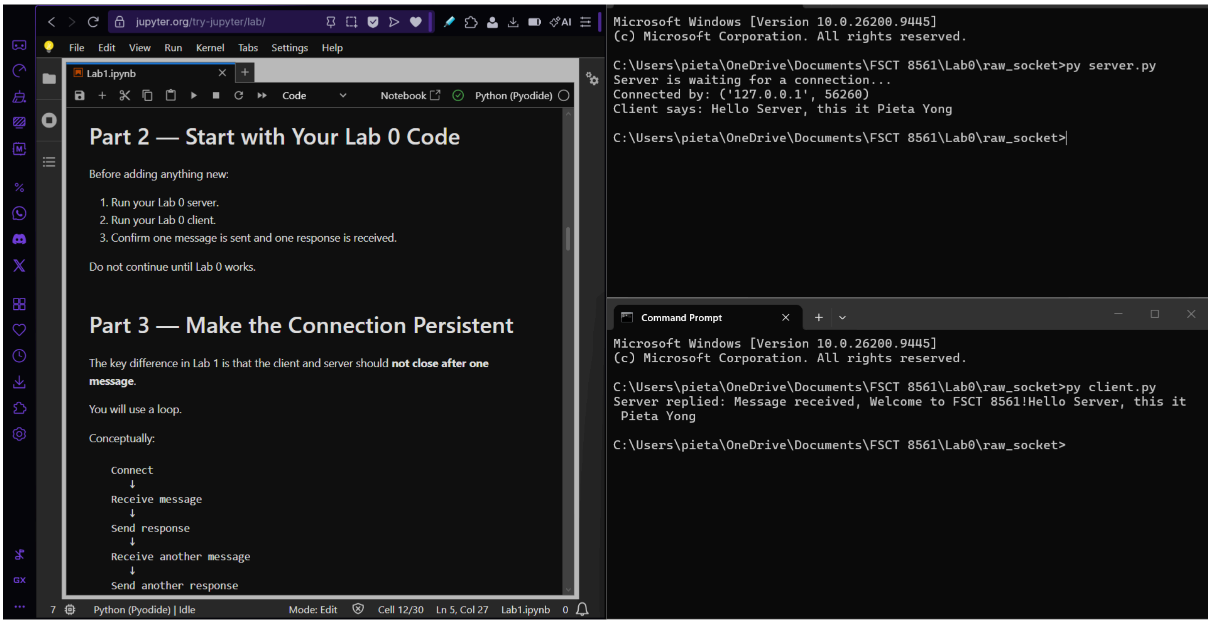
Part 2 - Lab 0



Part 2 — Start with Your Lab 0 Code

Before adding anything new:

1. Run your Lab 0 server.
2. Run your Lab 0 client.
3. Confirm one message is sent and one response is received.

Do not continue until Lab 0 works.

Part 3 — Make the Connection Persistent

The key difference in Lab 1 is that the client and server should **not close after one message**.

You will use a loop.

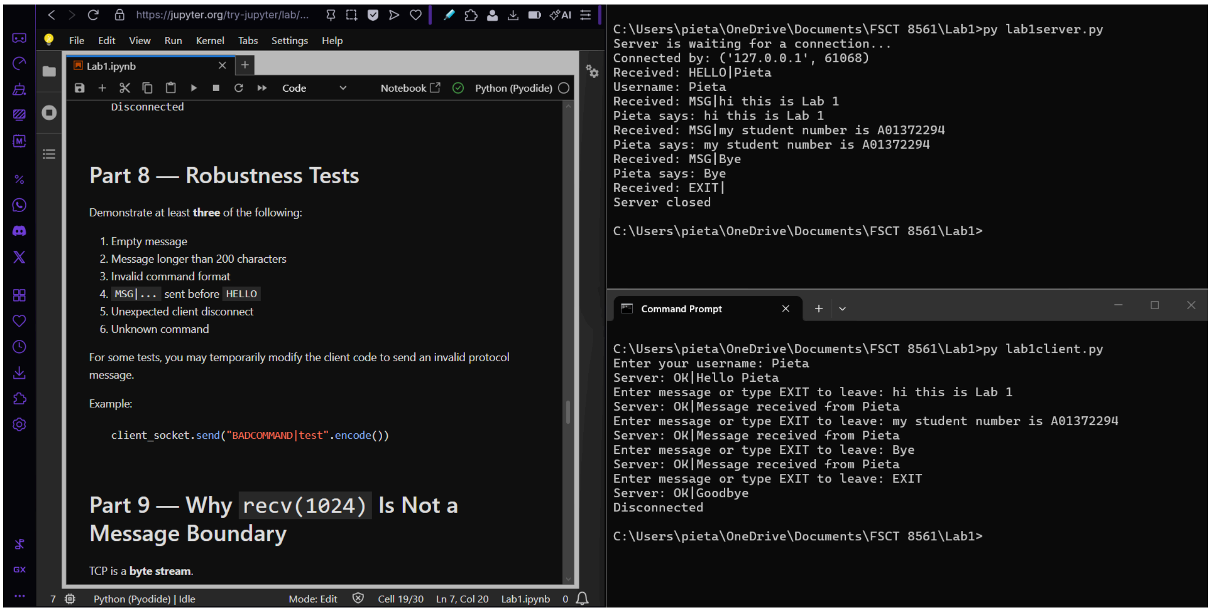
Conceptually:

```
Connect
 ↓
Receive message
 ↓
Send response
 ↓
Receive another message
 ↓
Send another response
```

```
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab0\raw_socket>py server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 56260)
Client says: Hello Server, this it Pieta Yong

C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab0\raw_socket>
```

```
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab0\raw_socket>py client.py
Server replied: Message received, Welcome to FSCT 8561!Hello Server, this it
 Pieta Yong

C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab0\raw_socket>
```

Part 7 - Test Normal Operation



```
Disconnected
```

Part 8 — Robustness Tests

Demonstrate at least **three** of the following:

1. Empty message
2. Message longer than 200 characters
3. Invalid command format
4. `MSG|...` sent before `HELLO`
5. Unexpected client disconnect
6. Unknown command

For some tests, you may temporarily modify the client code to send an invalid protocol message.

Example:

```
client_socket.send("BADCOMMAND|test".encode())
```

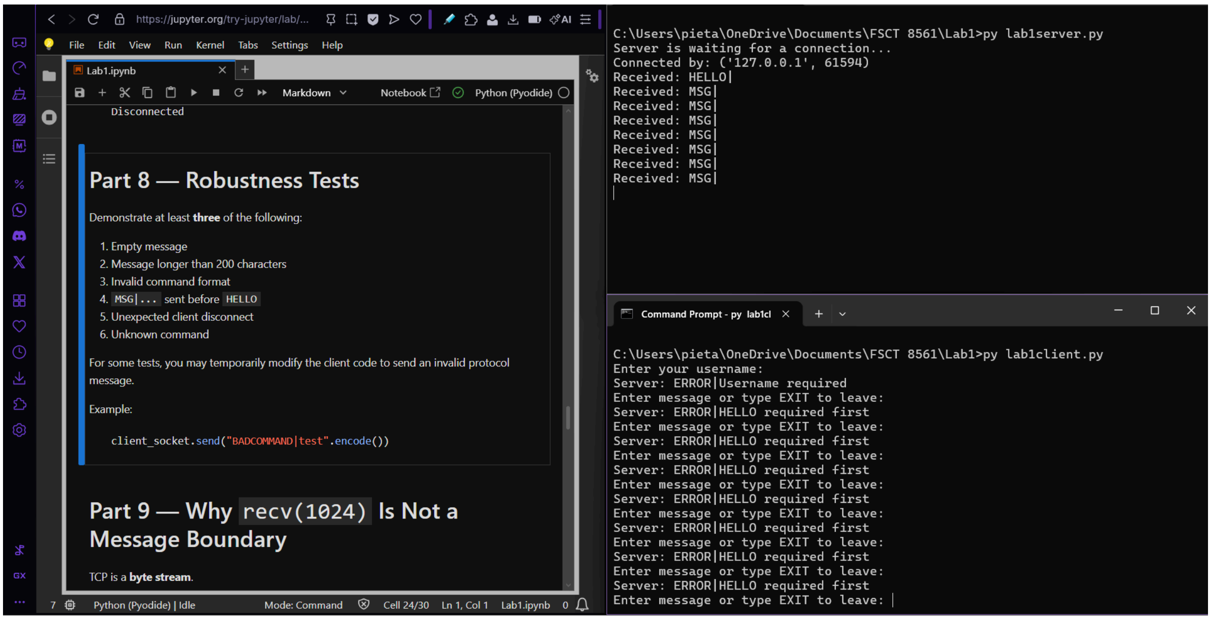
Part 9 — Why `recv(1024)` Is Not a Message Boundary

TCP is a **byte stream**.

```
C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab1>py lab1server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 61068)
Received: HELLO|Pieta
Username: Pieta
Received: MSG|hi this is Lab 1
Pieta says: hi this is Lab 1
Received: MSG|my student number is A01372294
Pieta says: my student number is A01372294
Received: MSG|Bye
Pieta says: Bye
Received: EXIT|
Server closed

C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab1>
```

```
C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab1>py lab1client.py
Enter your username: Pieta
Server: OK|Hello Pieta
Enter message or type EXIT to leave: hi this is Lab 1
Server: OK|Message received from Pieta
Enter message or type EXIT to leave: my student number is A01372294
Server: OK|Message received from Pieta
Enter message or type EXIT to leave: Bye
Server: OK|Message received from Pieta
Enter message or type EXIT to leave: EXIT
Server: OK|Goodbye
Disconnected

C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab1>
```

Part 8 - Empty Message


https://jupyter.org/try-jupyter/lab/...
File Edit View Run Kernel Tabs Settings Help
Lab1.ipynb
Markdown
Notebook
Python (Pyodide)
Python (Pyodide) | Idle
Mode: Command
Cell 24/30
Ln 1, Col 1
Lab1.ipynb
Command Prompt - py lab1cl

```
Disconnected
```

Part 8 — Robustness Tests

Demonstrate at least **three** of the following:

1. Empty message
2. Message longer than 200 characters
3. Invalid command format
4. `MSG|...` sent before `HELLO`
5. Unexpected client disconnect
6. Unknown command

For some tests, you may temporarily modify the client code to send an invalid protocol message.

Example:

```
client_socket.send("BADCOMMAND|test".encode())
```

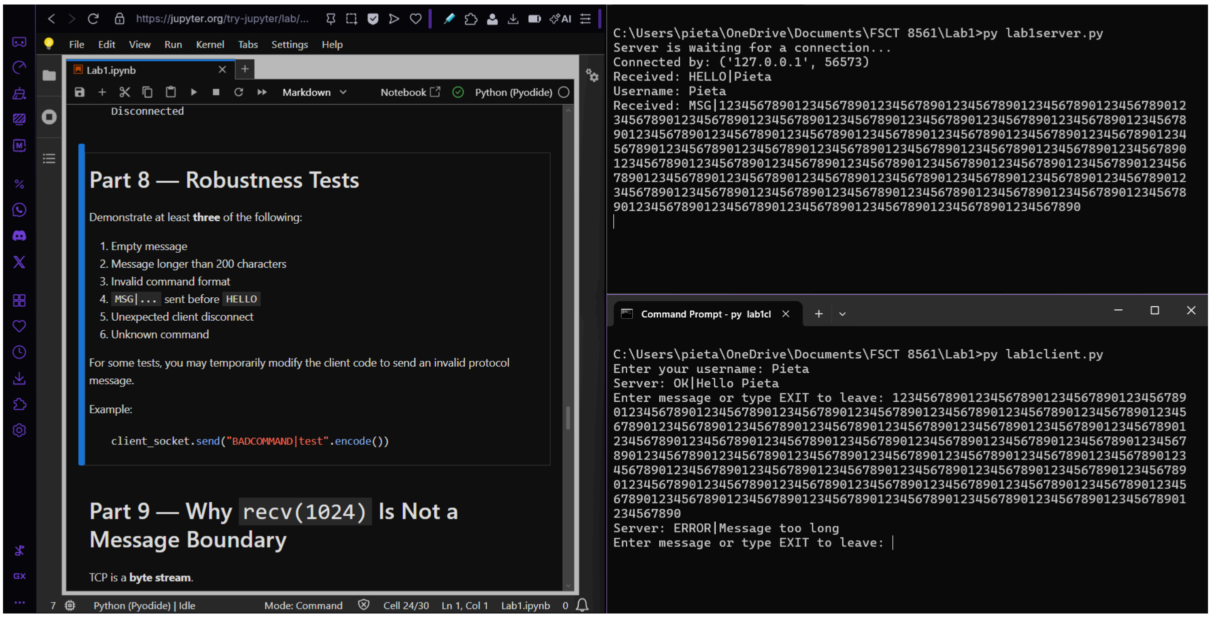
Part 9 — Why `recv(1024)` Is Not a Message Boundary

TCP is a **byte stream**.

```
C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab1>py lab1server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 61594)
Received: HELLO|
Received: MSG|
Received: MSG|
Received: MSG|
Received: MSG|
Received: MSG|
Received: MSG|
Received: MSG|
```

```
C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab1>py lab1client.py
Enter your username:
Server: ERROR|Username required
Enter message or type EXIT to leave:
Server: ERROR|HELLO required first
Enter message or type EXIT to leave:
Server: ERROR|HELLO required first
Enter message or type EXIT to leave:
Server: ERROR|HELLO required first
Enter message or type EXIT to leave:
Server: ERROR|HELLO required first
Enter message or type EXIT to leave:
Server: ERROR|HELLO required first
Enter message or type EXIT to leave:
Server: ERROR|HELLO required first
Enter message or type EXIT to leave:
Server: ERROR|HELLO required first
Enter message or type EXIT to leave:
```

Part 8 - Longer than 200 characters


https://jupyter.org/try-jupyter/lab/...
File Edit View Run Kernel Tabs Settings Help
Lab1.ipynb
Markdown
Notebook
Python (Pyodide)
Python (Pyodide) | Idle
Mode: Command
Cell 24/30
Ln 1, Col 1
Lab1.ipynb
Command Prompt - py lab1cl

```
Disconnected
```

Part 8 — Robustness Tests

Demonstrate at least **three** of the following:

1. Empty message
2. Message longer than 200 characters
3. Invalid command format
4. `MSG|...` sent before `HELLO`
5. Unexpected client disconnect
6. Unknown command

For some tests, you may temporarily modify the client code to send an invalid protocol message.

Example:

```
client_socket.send("BADCOMMAND|test".encode())
```

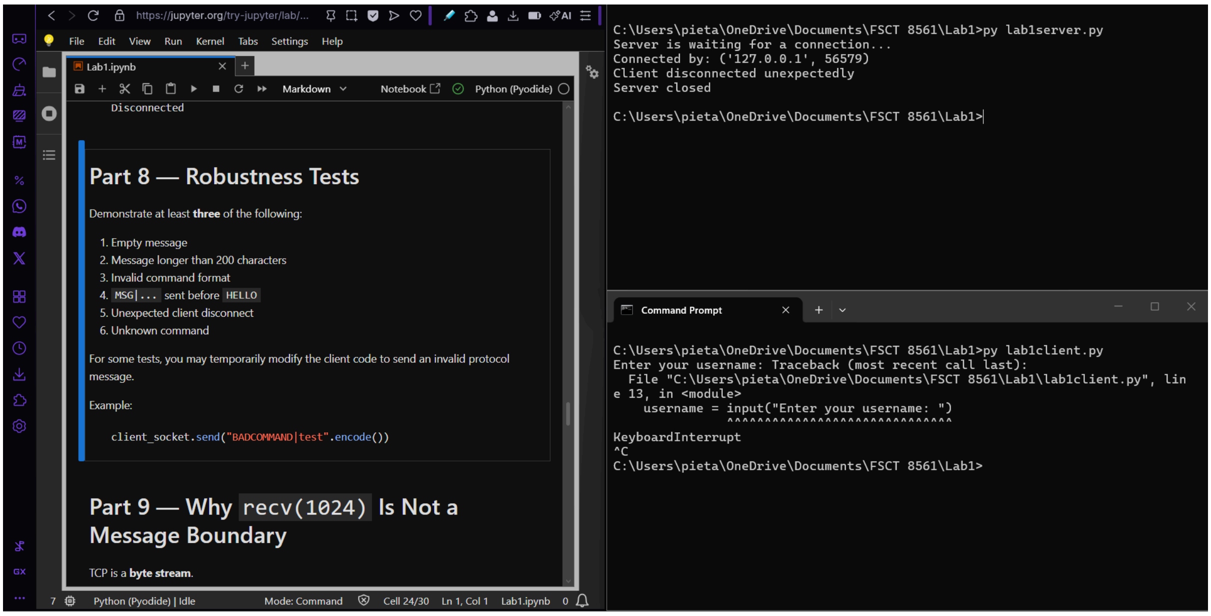
Part 9 — Why `recv(1024)` Is Not a Message Boundary

TCP is a **byte stream**.

```
C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab1>py lab1server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 56573)
Received: HELLO|Pieta
Username: Pieta
Received: MSG|123456789012345678901234567890123456789012345678901234567890123456789012
345678901234567890123456789012345678901234567890123456789012345678901234567890
901234567890123456789012345678901234567890123456789012345678901234567890123456
567890123456789012345678901234567890123456789012345678901234567890123456789012
123456789012345678901234567890123456789012345678901234567890123456789012345678
789012345678901234567890123456789012345678901234567890123456789012345678901234
345678901234567890123456789012345678901234567890123456789012345678901234567890
901234567890123456789012345678901234567890123456789012345678901234567890
```

```
C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab1>py lab1client.py
Enter your username: Pieta
Server: OK|Hello Pieta
Enter message or type EXIT to leave: 1234567890123456789012345678901234567890123456789
0123456789012345678901234567890123456789012345678901234567890123456789012345
6789012345678901234567890123456789012345678901234567890123456789012345678901
2345678901234567890123456789012345678901234567890123456789012345678901234567
8901234567890123456789012345678901234567890123456789012345678901234567890123
4567890123456789012345678901234567890123456789012345678901234567890123456789
0123456789012345678901234567890123456789012345678901234567890123456789012345
6789012345678901234567890123456789012345678901234567890123456789012345678901
234567890
Server: ERROR|Message too long
Enter message or type EXIT to leave:
```

Part 8 - Unexpected Client Disconnect



```
Disconnected
```

Part 8 — Robustness Tests

Demonstrate at least **three** of the following:

1. Empty message
2. Message longer than 200 characters
3. Invalid command format
4. `MSG|...` sent before `HELLO`
5. Unexpected client disconnect
6. Unknown command

For some tests, you may temporarily modify the client code to send an invalid protocol message.

Example:

```
client_socket.send("BADCOMMAND|test".encode())
```

Part 9 — Why `recv(1024)` Is Not a Message Boundary

TCP is a **byte stream**.

```
C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab1>py lab1server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 56579)
Client disconnected unexpectedly
Server closed

C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab1>
```

```
C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab1>py lab1client.py
Enter your username: Traceback (most recent call last):
  File "C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab1\lab1client.py", lin
e 13, in <module>
    username = input("Enter your username: ")
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt
^C
C:\Users\pieta\OneDrive\Documents\FSCT 8561\Lab1>
```

Part 10

1. **What happens if the server crashes while the client is connected?**
    a. The client may receive an error or an empty response when it tries to send or receive data.
2. **How could the server be extended to handle multiple clients?**
    a. Threading - Connecting clients by a separate thread, allowing multiple clients to communicate with the server at the same time.
3. **Why can `recv(1024)` split messages unexpectedly?**
    a. TCP treats data as a byte stream and not an individual message.
4. **Why does TCP reliability not mean the application is secure?**
    a. TCP is delivered reliably and in order, this does not provide security.
5. **What new attack surfaces are created by adding usernames and protocol commands?**
    a. Usernames and commands are a user controlled input, an attacker could use long usernames, commands, or impersonate another username.
6. **How could a malicious client abuse the protocol?**
    a. Wasting server resources or cause it to crash by: spamming messages, invalid commands, purposefully disconnect unexpectedly
7. **How would you add authentication?**
    a. Make it so that the users have to provide credentials like usernames and passwords for the server to verify before allowing the user to send messages.
8. **How would you add encryption?**
    a. Use TLS connected so that the traffic is encrypted.
9. **What validation is performed in your implementation?**
    a. The server checks whether the message contains the "| ", requires a username, and rejects empty messages.
10. **What additional mitigations would be needed for a real deployment?**
    a. Authentication
    b. TLS Encryption
    c. Secure credential storage